Tarea 2 - 2026: Diseño de un Sistema de Microservicios para una Plataforma de Comercio Electrónico

En esta tarea, se mantendrán los grupos de la tarea 1, y se diseñarán e implementarán un sistema de microservicios para **"ShopExpress"**, una plataforma ficticia de comercio electrónico y procesamiento de pedidos. El objetivo es aplicar los conceptos de bases de datos políglotas, consistencia, escalabilidad y patrones de diseño, justificando cada decisión.

Contexto de la Realidad Ficticia

"ShopExpress" es una empresa ficticia dedicada a la venta y entrega de productos en línea. Para manejar el alto volumen de pedidos y consultas de catálogo, se ha optado por una arquitectura de microservicios.

Los microservicios clave son:

- **Servicio de Usuarios**: Gestiona los perfiles de los clientes, direcciones de envío e historial de compras.
- **Servicio de Catálogo e Inventario**: Maneja la información de los productos, categorías, precios y stock disponible.
- **Servicio de Pedidos y Pagos**: Procesa las transacciones de compra de productos y se comunica con otros servicios para completar la orden de compra.

Requerimientos y retos del Diseño

Para cada uno de los siguientes puntos, deben justificar sus decisiones de diseño basándose en los requerimientos de la plataforma.

1. **Requerimientos de Consistencia y Lectura/Escritura**:
    - **Lectura de catálogo y usuarios**: Debe ser extremadamente rápida y escalable. Se priorizará la **disponibilidad** y la **tolerancia a particiones**, aceptando la **consistencia eventual**.
    - **Escritura de pedidos y pagos**: Requiere alta consistencia. Un pedido debe ser único y no puede haber venta sin stock (sobreventa). Se priorizará la consistencia sobre la velocidad.
2. **Modelado de Datos (NoSQL)**:
    - Deben elegir al menos dos bases de datos **NoSQL** y asignarlas a los microservicios.

- Justifiquen las tecnologías elegidas, y cómo modelaron los datos de productos y usuarios. Decidan entre los patrones de datos `embebido` o `referencia` para relacionar la información.

3. **Patrones de Microservicios para Transacciones y Lógica de Negocio**:
   - `Patrón SAGA`: El proceso de creación y pago de un pedido es una transacción distribuida que involucra múltiples microservicios. Se sugiere implementar el patrón **SAGA** con el enfoque de **Orquestación** para asegurar la consistencia.
     - **Implementación**: El Servicio de Pedidos y Pagos debe actuar como un Orquestador Central que coordina la secuencia de pasos de la transacción. Detallen los pasos de la transacción exitosa y las transacciones de compensación en caso de fallo.
   - `Patrón Chain of Responsibility`: La lógica de negocio dentro del Servicio de Pedidos y Pagos es secuencial. La solicitud de pedido se sugiere que pase por una serie de validaciones y pasos. Se sugiere utilizar el patrón **Chain of Responsibility** para estructurar esta lógica.
     - **Implementación**: Diseñar una cadena de "manejadores" para la solicitud de pedido. Los manejadores podrían incluir: ValidadorDeDatos, ValidadorDeStock, ProcesadorDePago, etc.
4. **Patrones Avanzados y Privacidad (Opcional)**:
   - **Event Sourcing / CQRS / Privacidad**: Mencionar brevemente cómo podrían aplicarse Event Sourcing, CQRS o técnicas de seudonimización/cifrado de datos personales en esta arquitectura.
5. **Tecnologías de Despliegue**:
   - Se valorará usar Docker y Docker Compose y/o minikube.

Se puede utilizar los lenguajes de programación y frameworks: Python (FastAPI), Spring Boot, Node con Express, Net.

Descripción Detallada de los Endpoints a Diseñar

- **Servicio de Usuarios**:
  - `POST /api/usuarios`: Crea un nuevo usuario.

    - `GET /api/usuarios/{usuario_id}`: Recupera un usuario.
    - `GET /api/usuarios/exportar`: Exporta datos de usuarios en formato JSON/CSV con datos anonimizados. (Opcional)
- **Servicio de Catálogo e Inventario**:
    - `POST /api/productos`: Crea un nuevo producto.
    - `GET /api/productos/{producto_id}`: Recupera la información de un producto y su stock disponible.
- **Servicio de Pedidos y Pagos**:
    - `POST /api/pedidos`: Inicia la transacción **SAGA** y el proceso **Chain of Responsibility**

Guía de Entrega

Entrega de la Documentación (Git)

Se debe entregar en un único repositorio de Git, que contenga un archivo `README.md` con la siguiente información:

- **Justificación del Diseño**: Un apartado que explique las decisiones de diseño.
- **Diagrama de Arquitectura**: Un diagrama que muestre los microservicios, las bases de datos y las interacciones.
- **Diagrama de Flujo del Patrón SAGA**: Un diagrama que ilustra el flujo completo de la transacción.
- **Diagrama del Patrón Chain of Responsibility**: Un diagrama que muestre la secuencia de los manejadores dentro del Servicio de Reservas y Pagos.

Entrega del Código (Git)

El código fuente de los microservicios y los archivos de configuración de Docker (o Kubernetes si es el caso) deben estar organizados en el mismo repositorio.

- **Estructura de Directorios**: Los microservicios deben estar en directorios separados.
- **Archivos de Configuración**: Se deben incluir los archivos `docker-compose.yml y Dockerfiles en caso de utilizar docker`.
- **Comandos de Ejecución**: En el `README.md` deben figurar los comandos para construir, levantar y probar la aplicación, y toda la documentación de la solución. Colecciones postman o equivalentes que evidencien el testeo.

Fecha de entrega y defensa : jueves 5 y 12 de noviembre.

Los grupos tendrán un tiempo de presentación aproximado de 10 minutos, dónde mostraran demo del sistema, requerimientos implementados, arquitectura y opcionalmente podrá usar diapositivas.

Anexo: Ejemplos de Estructuras de Datos

Estructura de Datos de un Pedido (JSON)

```
{
  "pedido_id": "ord_98765",
  "usuario_id": "usr_12345",
  "estado": "PROCESANDO",
  "items": [
    {
      "producto_id": "prod_001",
      "cantidad": 2,
      "precio_unitario": 25.50
    }
  ],
  "total": 51.00,
  "pago": {
    "metodo": "TARJETA_CREDITO",
    "estado_pago": "APROBADO",
    "transaccion_id": "tx_456789"
  }
}
```

Estructura de Datos de Usuario (JSON)

```
{
  "id": "usr_12345",
  "tipo_documento": "DNI",
  "nro_documento": "12345678",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan.perez@example.com",
  "direcciones": [
    {
      "calle": "Av. Principal 123",
      "ciudad": "Montevideo",
      "codigo_postal": "11000"
    }
  ]

}
```